



# **Aplicação de Gestão de Unidade de Saúde**

**Aplicações Distribuídas**  
**Mestrado em Informática Médica**

**TP2**

**Braga, janeiro 2024**

**Docente: António Luís Sousa**

**Grupo 7:**

Mariana Ribeiro, pg54061;

Mariana Almeida, pg54062;

Madalena Passos, pg54023.

# Índice

|                                                       |    |
|-------------------------------------------------------|----|
| <b>1. Introdução</b>                                  | 3  |
| <b>2. Contextualização e Arquitetura da Aplicação</b> | 4  |
| <b>3. Definição dos Modelos</b>                       | 7  |
| 3.1. Utilizador                                       | 7  |
| 3.2. Utente                                           | 7  |
| 3.3. Familiar                                         | 8  |
| 3.4. Rececionista                                     | 8  |
| 3.5. Medico                                           | 8  |
| 3.6. FichaMedica                                      | 9  |
| 3.7. DadosFisiologicos                                | 9  |
| 3.8. Consulta                                         | 9  |
| 3.9. Exame                                            | 10 |
| 3.10. Medicamento                                     | 10 |
| 3.11. Prescricao                                      | 10 |
| <b>4. Implementação</b>                               | 11 |
| 4.1. Página LOGIN                                     | 12 |
| 4.2. Página LOGOUT                                    | 13 |
| 4.3. Repor PASSWORD                                   | 13 |
| 4.4. Menu UTENTE:                                     | 13 |
| 4.5. Menu FAMILIAR                                    | 15 |
| 4.6. Menu MÉDICO:                                     | 16 |
| 4.7. Menu RECECIONISTA:                               | 20 |
| <b>5. Conclusões e Trabalho Futuro</b>                | 23 |

## 1. Introdução

Atualmente, os sistemas distribuídos têm desempenhado um papel crucial no desenvolvimento de sistemas de *software*, promovendo a colaboração eficiente entre diferentes entidades.

Assim, no âmbito da disciplina de Aplicações Distribuídas, este trabalho centra-se na implementação de uma aplicação que pretende realizar a gestão e registo de cuidados de saúde de uma unidade de saúde. A aplicação desenvolvida conta com quatro tipos distintos de utilizadores, com funções e autorizações de utilização diferentes.

Para uma maior facilidade de implementação da aplicação e todos os seus constituintes foi utilizada a tecnologia *DJANGO*. Esta tecnologia proporciona uma ferramenta de *ORM* - Object-Relational Mapping - que permite a interação com a base de dados utilizando objetos em *Python*, facilitando a procura e gestão de informação. Por outro lado, o *DJANGO* utiliza um sistema de mapeamento de *URLs* que permite a definição de *URLs* de maneira clara e eficiente, bem como um sistema de *templates HTML* que promove um desenvolvimento de páginas *web* de forma mais clara e simples.

Este trabalho explorará a implementação prática da aplicação *DJANGO*, abordando aspetos como a arquitetura geral, o papel específico de cada tipo de utilizador e a interação entre as *views* desenvolvidas, os formulários utilizados e as páginas *HTML* criadas.

## 2. Contextualização e Arquitetura da Aplicação

A realização deste projeto tinha em vista o desenvolvimento de uma aplicação que permitisse a gestão de recursos de um centro prestador de cuidados de saúde, sendo esta destinada a quatro tipos de utilizadores distintos, os médicos, os rececionistas, os utentes e os familiares associados a estes. Para um desenvolvimento mais orientado desta aplicação, foi necessário, então, elaborar uma arquitetura que permitisse ter em conta como seria efetuado o acesso às informações pretendidas. Assim, obteve-se o esquema apresentado na Figura 1.

Através da Figura 1 pode ser observada a definição de cada um destes utilizadores, que possuem atributos próprios, como por exemplo o *num\_utente*, o *niss* e o género para os utentes, assim como atributos comuns entre si. Estes atributos partilhados são estabelecidos deste modo por serem considerados informação transversal importante a qualquer um dos utilizadores, fazendo parte deste grupo o *username* e a palavra-passe, o primeiro e último nome, o número de identificação (*cc*), os meios de contacto e a data de nascimento. Os dois primeiros atributos desta lista permitem que os utilizadores possam ter acesso à plataforma através de uma página de login comum.

Pode-se ainda destacar a presença de outras entidades, como a ficha médica, o médico, a consulta, a prescrição, o exame, os dados fisiológicos e, por fim, o medicamento. Estas entidades encontram-se relacionadas com os utilizadores mencionados e permitem que os mesmos possam consultar, adicionar ou editar informação de cada uma delas, dependendo, naturalmente, da sua função dentro da unidade de saúde e do que esta requer.

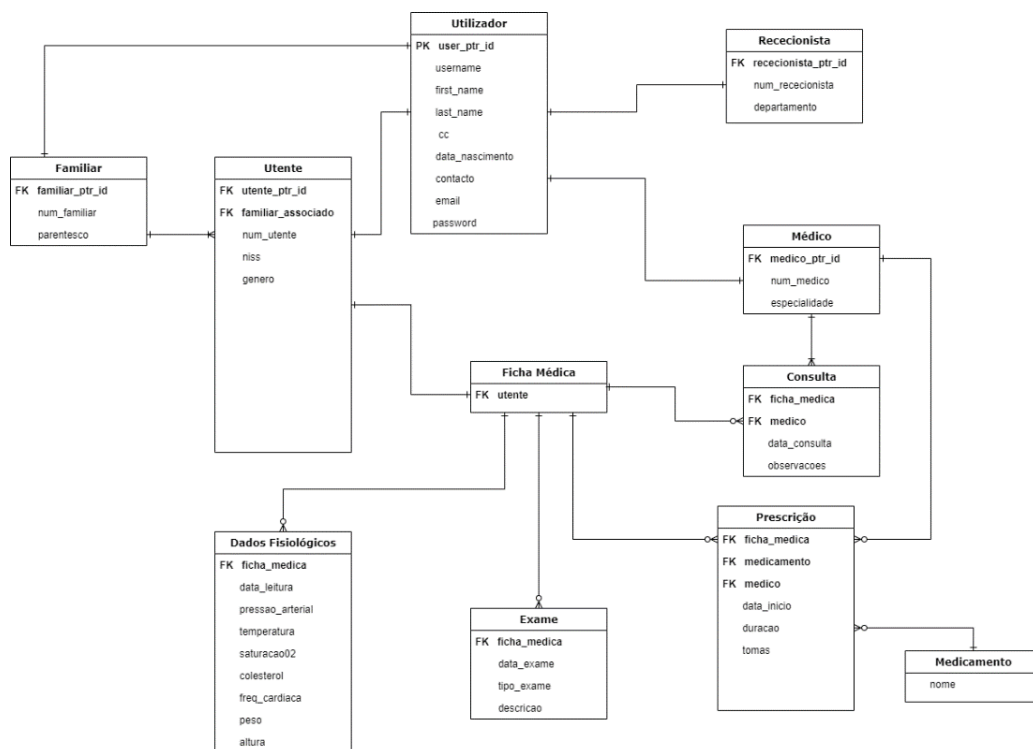


Figura 1- Arquitetura da Aplicação.

Observando, então, as relações entre cada entidade é possível entender o raciocínio que deu origem a esta arquitetura:

1. Existe uma tabela da qual constam atributos comuns a todos os utilizadores, sendo estabelecida, assim, uma relação de um para um entre esta e cada um dos utilizadores em causa, sendo que para cada utilizador o conjunto destas informações é única;
2. Um utente tem a si associado um familiar, como contacto de emergência, porém, o mesmo familiar poderá estar associado a múltiplos utentes em simultâneo;
3. Cada utente possui exclusivamente uma ficha médica designada especificamente para ele. Esta é composta por levantamentos de dados fisiológicos, consultas, prescrições e exames. É importante notar que cada utente pode ter zero ou mais de cada um destes tipos de registos nesta ficha, proporcionando flexibilidade na documentação dos cuidados de saúde;
4. Relativamente às prescrições, estas necessitam de ter um médico associado responsável por elas. Deste modo poderão existir médicos com múltiplas prescrições associadas, assim como nenhuma, se for o caso. De um modo muito idêntico, tem-se os medicamentos, que podem ter associados a si prescrições, ou não, havendo sempre, porém, um medicamento associado a cada prescrição;
5. No que toca às consultas, estas relacionam-se, para além das fichas médicas, com os médicos, uma vez que para existir uma consulta tem de haver um médico que a dê. Porém, um médico em início de carreira naquele local não terá dado ainda nenhuma consulta, enquanto médicos que se encontrem nesta unidade de saúde há mais tempo terão diversas consultas a si associadas.

Tendo estas relações em conta, foi necessário estabelecer 4 grupos de utilizadores, com diferentes permissões, cada um com o nome de um tipo de utilizador.

Começando por explorar as permissões do grupo Rececionista, este poderá efetuar as seguintes operações:

- Registar Utentes, Médicos e Familiares;
- Adicionar consultas à Ficha Médica de um determinado Utente, bem como alterar as mesmas;
- Consultar e editar informações pessoais de um determinado Utente;
- Consultar histórico de Exames e de Consultas de um determinado Utente;
- Consultar Utentes associados a um Familiar;
- Visualizar as Consultas dadas por cada Médico;
- Consultar o número de Consultas e Exames efetuadas num determinado dia;
- Consultar o histórico de Consultas;

- Alterar os dados pessoais de um Médico;
- Repor palavra-passe.

Relativamente aos Médicos, estes podem:

- Adicionar leituras de Dados Fisiológicos, Exames, Prescrições na Ficha Médica de um dado utente;
- Consultar as suas informações pessoais;
- Consultar histórico de Exames e de medicamentos prescritos a um determinado utente;
- Editar Consultas de um determinado utente;
- Consultar todos os Utentes associados a si;
- Consultar o histórico de Consultas e de Exames efetuados num dia;
- Consultar todos os Utentes com leituras de imc acima da média;
- Consultar todo o histórico de Prescrições e de Consultas;
- Repor palavra-passe.

No que toca aos Utentes, estes podem:

- Consultar as suas informações pessoais;
- Consultar as suas Prescrições, Exames, Consultas, Dados Fisiológicos;
- Consultar o seu Familiar associado;
- Alterar o seu Familiar associado;
- Repor palavra-passe.

Por fim, os Familiares podem:

- Consultar as suas informações pessoais;
- Consultar as informações pessoais do Utente a ele associado;
- Consultar Consultas, Prescrições, Exames e Dados Fisiológicos do Utente a ele associado;
- Repor palavra-passe.

Em conclusão, a arquitetura delineada para a gestão de informações de cuidados de saúde apresenta uma estrutura cuidadosamente planeada, refletindo a complexidade das relações entre as diversas entidades envolvidas. O estabelecimento das permissões para cada grupo de utilizadores foi projetado para satisfazer as necessidades específicas de cada função, promovendo eficiência operacional e assegurando a segurança e integridade dos dados.

### 3. Definição dos Modelos

Os *models* da aplicação foram definidos no ficheiro *models.pys*. Estes encontram-se implementados como subclasse do *django.db.models.Model*, e incluem campos, métodos e metadados. Com base nestes modelos, o *Django* irá gerar automaticamente uma API de acesso à base de dados.

Com efeito, para o desenvolvimento da aplicação foram criados modelos referentes aos utilizadores: *Utilizador*, *Utente*, *Familiar*, *Rececionista* e *Médico*, bem como aos dados clínicos: *FichaMedica*, *DadosFisiologicos*, *Consulta*, *Exame*, *Medicamento* e *Prescricao*.

A seguir, é fornecida uma análise detalhada de cada modelo.

#### 3.1. Utilizador

O modelo *Utilizador* desempenha um papel muito importante na organização hierárquica da aplicação, representando uma entidade genérica de utilizador. Esta classe herda da classe *User* do *Django* atributos comuns a todos os utilizadores, nomeadamente, *first\_name*, *last\_name*, *email*, *username* e *password*.

Foram ainda definidos atributos como o número de cartão de cidadão (*cc*) e contacto (*contacto*). O tipo de campo definido para estes foi o *CharField*, uma vez que correspondem a dados alfanuméricos. De notar que o número de *cc* para cada utilizador deve ser único, ativando-se, portanto, a opção *unique=True*. Definiu-se ainda o atributo data de nascimento (*data\_nascimento*), sendo o tipo de campo um *DateField*.

#### 3.2. Utente

O modelo *Utente* é uma extensão do modelo *Utilizador* e representa um utente no contexto de uma aplicação para gestão e registo de cuidados de saúde. Para além dos atributos herdados do modelo *Utilizador*, apresenta ainda um número de utente (*num\_utente*) definido como *CharField* e com a opção *unique=True*, um número de identificação da segurança social (*NISS*) não obrigatório (*null=True*), único (*unique=True*) e o tipo de campo definido como *CharField*. Contém ainda um campo não obrigatório (*null=True*) para armazenar a informação do género do utente (*genero*) e um campo *familiar\_associado* que constitui uma *ForeignKey* que associa o utente a um objeto do modelo *Familiar*. Esta relação pode ser não obrigatória, isto é, um utente pode não ter um familiar associado, definindo-se assim *null=True* e *blank=True*.

Com o intuito de ordenar as instâncias do modelo com base no número de utente, definiu-se a classe *Meta* para o modelo com a configuração *ordering = ('num\_utente',)*.

Para além disso, implementou-se um método *\_\_str\_\_* que retorna uma representação de *string* do objeto *Utente* com o nome, apelido e número de utente.

### 3.3. Familiar

O modelo *Familiar* é também uma extensão do modelo *Utilizador* e representa um familiar no mesmo contexto. Para além dos atributos que herda do modelo *Utilizador*, apresenta também um número de familiar (*num\_familiar*) único e o atributo *parentesco*, ambos definidos como *CharField*.

De forma análoga ao modelo anterior, também possui uma classe *Meta*, de forma a ordenar as instâncias com base no número de familiar e ainda um método *\_\_str\_\_* que retorna em *string* o nome, apelido e número de familiar do objeto *Familiar*.

### 3.4. Rececionista

De forma muito semelhante aos modelos anteriormente explicados, o modelo *Rececionista* adota os atributos do modelo *Utilizador*. Além destes, é ainda composto por atributos definidos como *CharField* referentes ao número de rececionista (*num\_rececionista*) que constitui o seu identificador único, e ao departamento a que está associado (*departamento*).

Mais uma vez, de forma análoga aos modelos anteriores, é implementada uma classe *Meta* que ordena as instâncias com base no número de rececionista e ainda um método *\_\_str\_\_* que retorna em *string* o nome, apelido e número de rececionista do objeto *Rececionista*.

### 3.5. Medico

O modelo *Medico* herda também os atributos comuns a todos os utilizadores, acrescentando ainda detalhes específicos como o identificador único representado pelo número de médico (*num\_medico*) e ainda a *especialidade* que guarda a informação sobre a especialidade do médico. Ambos os atributos são definidos como *CharField*, possuindo um limite máximo de caracteres digitados.

Da mesma forma, apresenta uma classe *Meta* que ordena as instâncias com base no número de médico e ainda um método *\_\_str\_\_* que retorna em *string* o nome, apelido e número de médico do objeto *Medico*.



### 3.6. FichaMedica

O modelo *FichaMedica* representa a ficha médica associada a um determinado utente. Desta forma, possui um atributo *utente* que designa uma relação *OneToOneField* referente ao modelo *Utente*. Este tipo de relação implica que cada instância do modelo *FichaMedica* esteja apenas associada a uma única instância do modelo *Utente* e vice-versa. Assim, admite-se que uma ficha médica apenas pode estar associada a um utente, e por sua vez, um utente apenas pode possuir uma ficha médica.

Adicionalmente, o método `__str__` é implementado para fornecer uma representação de *string* do objeto *FichaMedica*. Essa representação inclui o número de utente e o nome do utente associado à ficha médica.

### 3.7. DadosFisiologicos

O modelo *DadosFisiologicos* permite armazenar dados importantes sobre as condições fisiológicas de um utente ao longo do tempo. Neste sentido, apresenta como *DateField* a data em que as leituras foram realizadas (*data\_leitura*) e os atributos *pressao\_arterial*, *temperatura*, *saturacaoO2*, *colesterol*, *freq\_cardiaca*, *peso* e *altura* como *DecimalField*. De notar que todos os atributos decimais apresentam os parâmetros *null=True* e *blank=True*, que permitem que os campos a preencher e sejam opcionais e possam ser guardados na base de dados como nulos.

Para além dos atributos referidos, apresenta ainda o campo *ficha\_medica* que representa uma *ForeignKey* referente ao modelo *FichaMedica*. Isto implica que cada instância do modelo esteja associada a uma única instância do modelo *FichaMedica*, permitindo que uma leitura de dados fisiológicos seja registada numa determinada ficha médica, por sua vez associada a um dado utente.

Por fim, o modelo possui ainda a classe *Meta* para ordenar as instâncias por data de leitura e ainda um método `__str__` para fornecer uma representação de *string* do objeto *DadosFisiologicos*.

### 3.8. Consulta

O modelo *Consulta* representa informações relacionadas a uma consulta médica. Desta forma foram definidos os atributos *data\_consulta* do tipo *DateTimeField* e *observacoes* do tipo *DateTimeField* que designam a data e hora da consulta e as observações facultativas sobre a mesma, respetivamente. Para além disso, possui o campo *fichamedica* que, há semelhança do

modelo anterior, representa uma *ForeignKey* referente ao modelo *FichaMedica*, indicando que cada consulta está associada a uma determinada ficha médica.

Da mesma forma, existe também um campo *medico* que representa uma *ForeignKey* referente ao modelo *Medico*. Com efeito, cada consulta deverá estar associada a um determinado médico.

A classe implementa ainda um método `__str__` que fornece uma representação de string do objeto Consulta. Esta representação inclui o nome do médico, o nome do utente e a data/hora da consulta.

### 3.9. Exame

O modelo *Exame* representa informações sobre exames médicos realizados pelos utentes. Desta forma, foram definidos como atributos a data e hora a que o exame foi realizado (*data\_exame*), o tipo de exame (*tipo\_exame*) e uma descrição opcional (*descrição*). Apresenta ainda o campo *fichamedica* que representa uma *ForeignKey* referente ao modelo *FichaMedica*, indicando que cada exame está associado a uma determinada ficha médica. A opção *null=False* não permite que o campo *fichamedica* seja nulo, indicando que um exame deve estar obrigatoriamente associado a uma determinada ficha médica.

Adicionalmente, foi implementado o método `__str__` para fornecer uma representação de *string* do objeto Exame. Esta representação inclui informações sobre o tipo de exame, data/hora do exame e uma breve descrição.

### 3.10. Medicamento

O modelo *Medicamento* representa informações sobre diferentes medicamentos disponíveis no sistema. Desta forma, foi definido o atributo *nome* com campo do tipo *CharField* que fornece opções predefinidas de nomes de medicamentos.

Adicionalmente, é implementando o método `__str__` para fornecer uma representação de *string* do objeto com base no nome do medicamento.

### 3.11. Prescricao

Por fim, o modelo *Prescricao* é fundamental para registar informações sobre as prescrições médicas realizadas para utentes específicos. Desta forma, foram definidos como atributos a data em que a prescrição foi iniciada (*data\_inicio*), a duração da administração do medicamento (*duracao*) e informação sobre a dosagem do medicamento (*tomas*).

Para além dos atributos referidos, a classe possui também um campo *fichamedica* que representa uma *ForeignKey* referente ao modelo *FichaMedica*, que indica que cada prescrição está associada a uma determinada ficha médica. Apresenta, ainda, o campo *medicamento* que também representa uma *ForeignKey* associada ao modelo *Medicamento*. Assim, cada prescrição deverá estar associada a um medicamento em específico, obrigatoriamente. Para além disso, o campo *medico* designa outra *ForeignKey* associada ao modelo *Medico*, dando a indicação do médico que realizou a prescrição.

Por fim, de forma semelhante a todos os modelos referidos anteriormente, é implementado o método `__str__` que fornece uma representação em *string* com a data de início, duração e tomas do objeto *Prescricao*.

## 4. Implementação

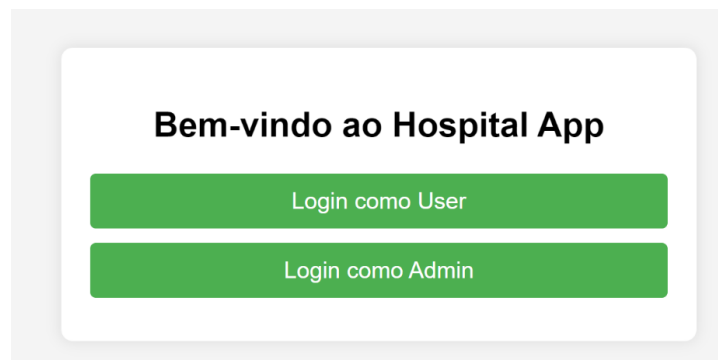
Para implementar a aplicação descrita acima utilizando os modelos de dados também já mostrados, foi necessário o desenvolvimento de *views* para cada funcionalidade mencionada. A cada *view* foi associada uma *template HTML* para mostrar de forma mais intuitiva a informação retornada pelos métodos.

No caso de *views* que necessitam de informação dada pelo utilizador foi também necessário desenvolver formulários com os campos respetivos para preenchimento. Adicionalmente, foram também definidos os *urls* das páginas *HTML* de forma a ser mais simples o redirecionamento e gestão dos dados.

Inicialmente, quando iniciado o servidor é mostrada a página *home*, mostrada na Figura 2, que permite ao utilizador escolher se pretende entrar como *User* ou *Admim*.

Se o utilizador escolher a opção de *Login* como *Admin*, vai ser redirecionado para a página de *Login* do *Django Administration*. Pelo contrário, se escolher fazer *Login* como *User* vai ser direcionado para a página de *Login* da Aplicação, mostrada mais a frente. Posteriormente, foram feitos diferentes menus para os diferentes utilizadores que vão ser explicados em mais detalhe.

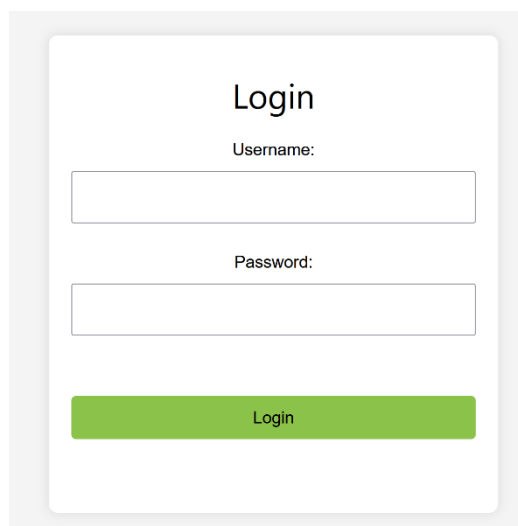
Por outro lado, foram também desenvolvidas as páginas de *Logout* e de repor *Password*, mostrados a seguir.



**Figura 2-**Página *home* da Aplicação.

#### 4.1. Página LOGIN

A página de *Login* é comum a todos os utilizadores. Esta página e *view* associada verifica se o *username* e *password* estão corretos, redireciona para o menu correspondente. Este redireccionamento é feito através dos grupos de utilizadores definidos: Utentes, Familiares, Médicos e Rececionistas. Caso o *username* pertença, por exemplo, ao grupo de utilizadores Utentes, o utilizador vai ser direcionado para o menu Utente.



**Figura 3-**Página de Login para todos os utilizadores.

Caso o *username* e *password* correspondam a um utilizador *superuser*, este vai ser redirecionado para a página *Django Administration*. No caso do *username* e *password* não corresponderem a nenhum utilizador, é visualizada uma mensagem de erro: “*Username* ou *password* incorretos. Tente novamente.”.

## 4.2. Página LOGOUT

De igual forma, a funcionalidade de *Logout* é comum a todos os utilizadores. Esta ferramenta utiliza a *view log\_out* que permite encerrar a sessão na *app* e voltar à página inicial de *login*, apresentando uma mensagem: “*Logout* bem sucedido”.

## 4.3. Repor PASSWORD

A função para repor *password* é também visível em todos os menus dos utilizadores. Foi definida então, para cada utilizador, uma *view repor\_pwd [tipo de utilizador]*, que utiliza um formulário para inserir a nova *password*. Após ser mudada a palavra-passe, a *view* redireciona para a *template HTML* do menu correspondente ao utilizador.

## 4.4. Menu UTENTE:

As funcionalidades do menu Utente centram-se maioritariamente na consulta dos seus dados pessoais, como já referido anteriormente. Assim, na Figura 4, mostra-se então a interface gráfica do menu Utente.



Figura 4-Funcionalidades do Menu Utente.

Como o utente pode, maioritariamente, consultar informação, todas as *views* são semelhantes. Utilizando o número do utente do utilizador passado como parâmetro no *request* das *views*, de forma a poder aceder às suas informações pessoais, quer no caso de consulta de dados pessoais, consultas, prescrições, exames, leituras de dados fisiológicos e familiar associado. Na Figura 5, abaixo, mostra-se a consulta de prescrições do utente.

Consulta de Prescrições por Utente

num\_utente:

Prescrições:

**Data Inicio** Jan. 10, 2024  
**Duração** 15 dias  
**Tomas** 5 5  
**Medicamento** Abece

[Voltar atrás](#) | [LogOut](#)

**Figura 5-**Consulta de Prescrições por Utente no Menu Utente.

Adicionalmente, o utente tem a possibilidade de alterar o seu familiar associado, através da introdução do *num\_familiar*. Neste caso, é feita a verificação de existência desse *num\_familiar* para assegurar que está a ser adicionado um familiar que exista.

## 4.5. Menu FAMILIAR

As funcionalidades do utilizador familiar estão apresentadas na Figura 6. O menu a que um familiar pode aceder inclui funcionalidades que permitem o acesso a dados pessoais do próprio familiar bem como informações sobre os utentes associados, nomeadamente, dados pessoais, consultas, prescrições, exames e dados fisiológicos realizados.



**Figura 6-**Funcionalidades do Menu Familiar.

De forma a permitir a visualização da página acima, foi implementada a *view menu\_familiar*, que verifica se o login do utilizador pertence efetivamente a um familiar. Em caso afirmativo, obtém o objeto Familiar associado e retorna o *template* HTML correspondente à Figura 6.

Como já referido anteriormente, o familiar possui apenas funcionalidades de consulta. Desta forma, foram implementadas *views* que permitem processar os pedidos do utilizador, manipulando e acedendo a informações contidas no banco de dados. Assim, todas as *views* implementadas funcionam de forma muito semelhante.

Por exemplo, a *view todas\_consultas\_ut\_fam*, inicialmente, irá verificar se o utilizador se encontra autenticado. Posteriormente, irá obter o objeto Familiar e recuperar os utentes a ele

associados. De seguida, irá a cada utente para obter as consultas médicas associadas. Por fim, retorna o *template* HTML observado abaixo com as informações sobre as consultas para cada utente.

**Consultas para Utente: Mariana Almeida**  
**Médico:** Joao Mendes, **Data/Hora:** Jan. 8, 2024, 11:32 a.m., **Observações:** Consulta Rotina  
**Médico:** Joao Mendes, **Data/Hora:** Jan. 9, 2024, 7:47 p.m., **Observações:** Consulta Urgente  
**Médico:** Nana Ribeiro, **Data/Hora:** Jan. 13, 2024, 11:44 a.m., **Observações:** Consulta Rotina  
**Médico:** Nana Ribeiro, **Data/Hora:** Jan. 13, 2024, 11:55 a.m., **Observações:** Consulta Urgente  
**Médico:** Dr. António Rodrigues, **Data/Hora:** Jan. 15, 2024, 6 p.m., **Observações:** Consulta Rotina  
**Médico:** Dr. Eduardo Fernandes, **Data/Hora:** Jan. 17, 2024, 2:05 p.m., **Observações:** Consulta Rotina

**Consultas para Utente: Mada Passos**  
Não há consultas para este utente.

**Consultas para Utente: Raquel Amil**  
**Médico:** Joao Mendes, **Data/Hora:** Jan. 13, 2024, 11:47 a.m., **Observações:** Agendamento  
Voltar atrás | LogOut

Figura 7-Consultas para cada um dos Utente associados.

## 4.6. Menu MÉDICO:

Como já referido anteriormente, as funcionalidades do utilizador médico são as apresentadas na Figura 8. Inicialmente, após o *login* por parte do utilizador, é verificado se este utente pertence ao grupo Medicos e, em caso afirmativo, é redirecionado para a página *HTML* correspondente, bem como para a *view menu\_medico*.

**Menu do Médico**  
Adicionar Leituras  
Adicionar Exames  
Adicionar Prescrições  
Consultar Informações Pessoais  
Consultar Exames de um Utente  
Alterar Consultas  
Meus Utentes  
Consultar Medicamentos Prescritos a um Utente  
Consultas Num Dia  
Exames Num Dia  
Consultar Utentes com IMC Acima da Média  
Histórico de Prescrições  
Histórico de Consultas

Figura 8-Funcionalidades do Menu Médico.



Assim, o utilizador do tipo médico, no geral, tem funcionalidades de consulta de dados relacionados a si, nomeadamente os seus dados pessoais, as suas consultas, os seus utentes (utentes com consultas dadas por esse médico), histórico de consultas e prescrições e os medicamentos e exames prescritos a um utente específico.

Estas últimas funcionalidades funcionam da seguinte forma: o utilizador introduz o número de utente a procurar, e caso o utente exista, são listados os objetos prescrição ou exames com ligação a este utente. Na Figura 9, mostra-se a *template html* que permite a procura de exames por utente.

A interface apresenta um cabeçalho verde com o título 'Consulta de Exames por Utente'. Abaixo, há um formulário com o rótulo 'num\_utente:' e o valor 'U1'. Um botão 'Procurar' está ao lado. O resultado da consulta é exibido sob o título 'Exames:'. São listados dois exames: o primeiro com data 'Jan. 10, 2024, 11:17 a.m.', tipo 'Eco' e descrição 'mt bom'; o segundo com data 'Jan. 9, 2024, 10:57 p.m.', tipo 'Raio X' e descrição 'Exame'. No rodapé, há links para 'Voltar atrás' e 'LogOut'.

**Figura 9-** Consulta de Exames por Utente no Menu Médico.

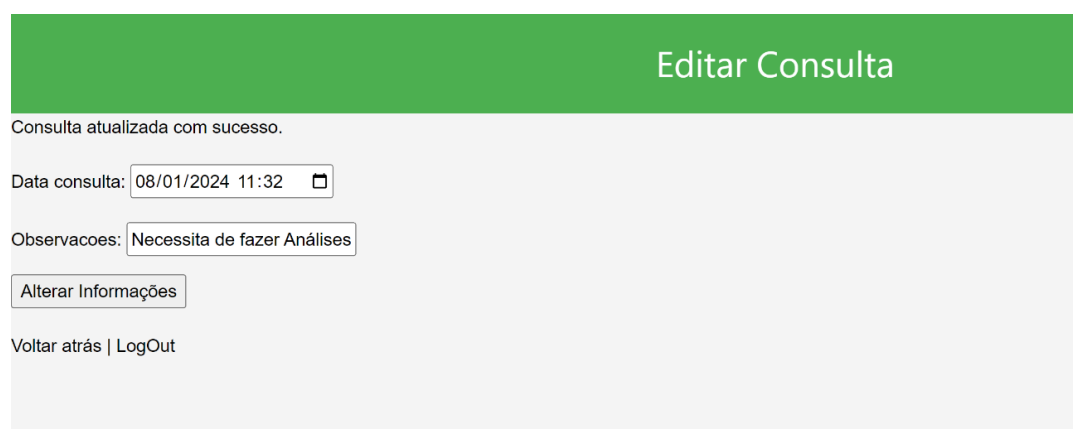
Adicionalmente, este utilizador pode adicionar leituras de dados fisiológicos, prescrições ou exames na ficha médica de um Utente. Todas estas funcionalidades necessitam que sejam introduzidos, através de um formulário na página *html*, parâmetros como os atributos do objeto e o número de utente, para procura do objeto a que esse utente corresponde. Posteriormente, são então encontrados o objeto utente e a sua ficha médica e adicionada uma consulta/exame/prescrição à mesma.

A interface tem um cabeçalho verde com o título 'Menu Médico - Registo de Prescrições'. O formulário principal, intitulado 'Adicionar Prescrições', contém campos para: 'Medicamento:' (menu suspenso com 'CEGRIPE' selecionado), 'Data inicio:' (16/01/2024 com ícone de calendário), 'Duracao:' (15 dias), 'Tomas:' (8h/8h) e 'Num utente:' (U1). Um botão 'Adicionar Prescrição' está abaixo dos campos. No rodapé, há links para 'Voltar ao Menu Médico' e 'LogOut'.

**Figura 10-** Registos de Prescrições na ficha médica de um Utente no Menu Médico.

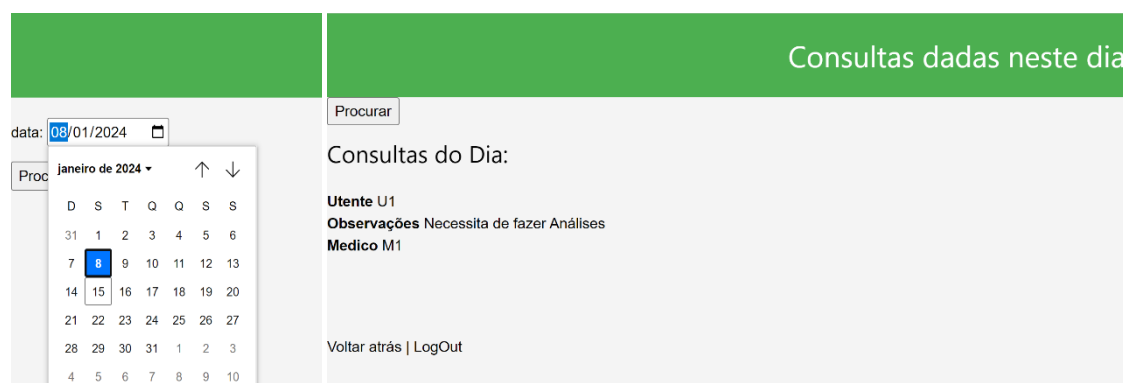
O utilizador médico tem também permissões que lhe permitem alterar uma consulta de um utente. Esta função pode ser utilizada para alterar as observações da consulta, já que o objeto Consulta funciona não só como histórico de consultas dadas, mas também agendamentos de consulta, dependendo da data introduzida no registo da mesma. Assim, a procura da consulta respetiva é feita pela data, como se mostra na Figura 11 e são substituídas as informações necessárias no parâmetro observações.

Caso a data da consulta não seja encontrada, é visualizada uma mensagem a informar que a consulta pretendida não existe. Caso a consulta exista, são efetuadas as alterações. A verificação das alterações, pode ser verificada na figura 12.



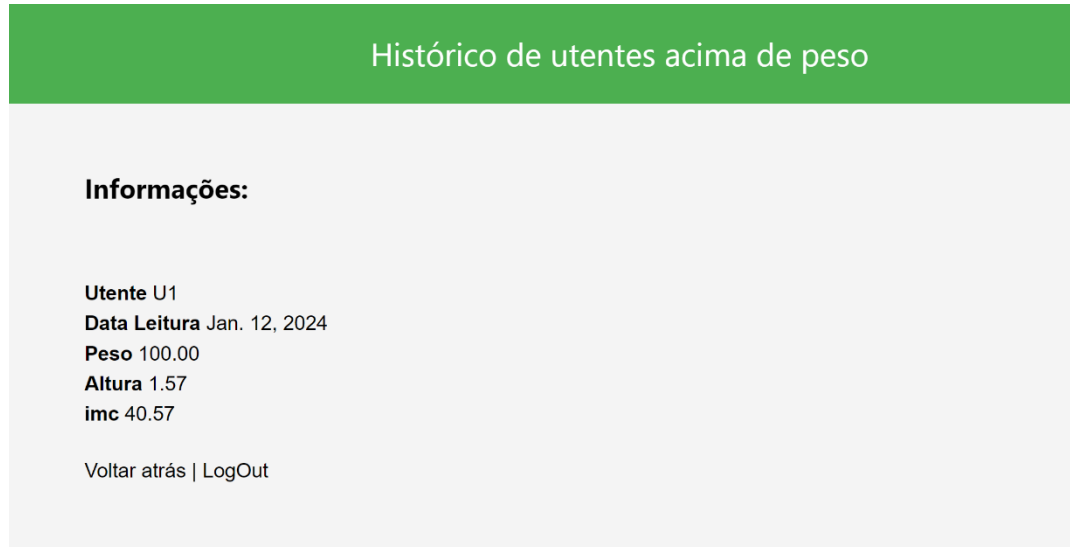
**Figura 11**-Editar consulta no Menu Médico.

Por fim, foram desenvolvidas algumas funcionalidades adicionais, nomeadamente a listagem de exames (de toda a clínica uma vez que o exame não tem correspondência a nenhum médico) ou consultas do médico numa data específica, introduzida num formulário da página *html* e passada como parâmetro de entrada na *view* correspondente. Esta última funcionalidade e o seu resultado pode ser observado na Figura 12.



**Figura 12**-Visualização das consultas dadas num dia específico no Menu Médico.

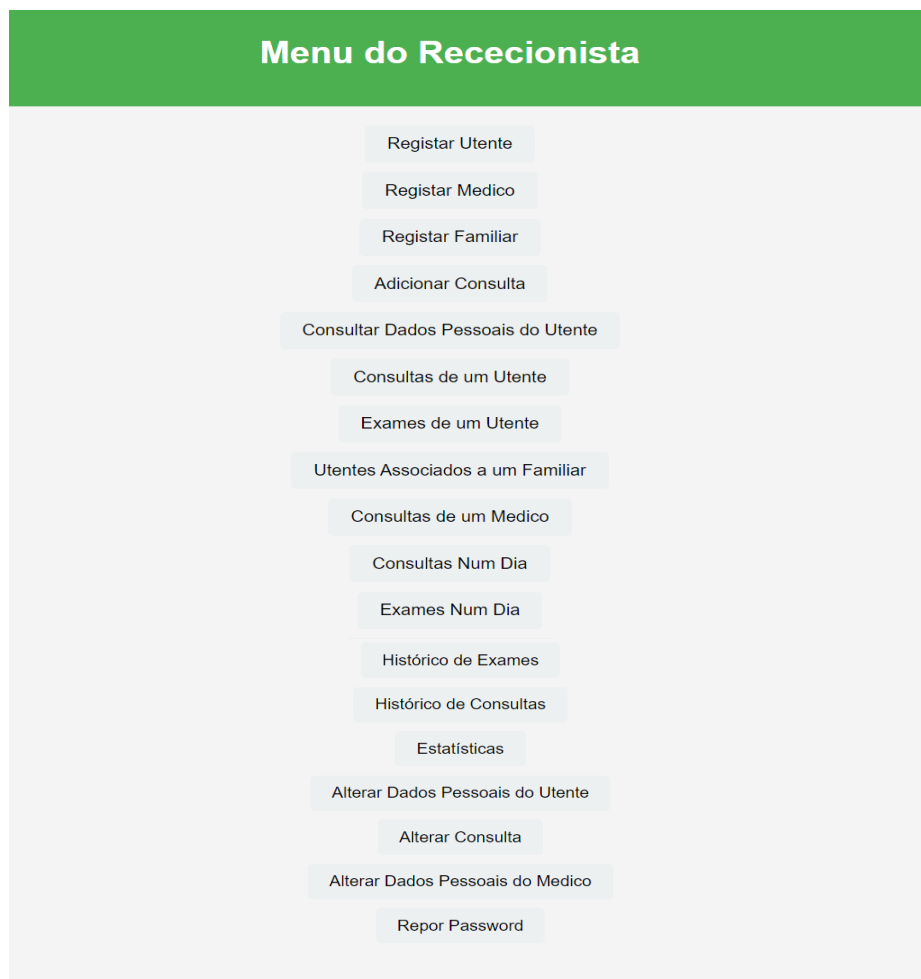
Foram ainda desenvolvidas uma *view* e página *html* que permitem que o médico visualize todas as leituras de dados fisiológicos dos seus utentes acima de peso. Esta procura utiliza o cálculo do IMC – índice de massa corporal - e encontra os utentes e o seu histórico de leituras com IMC acima do normal.



**Figura 13**-Histórico de Utentes acima do peso no Menu Médico.

## 4.7. Menu RECECIONISTA:

As funcionalidades do utilizador rececionista encontram-se expressas na Figura 14.



**Figura 14-** Funcionalidades do Menu Rececionista.

Conforme ilustrado na Figura 14, o utilizador rececionista possui funcionalidades de consulta, registo de utilizadores e consultas e possibilita ainda a alteração de alguns campos pessoais de um determinado médico ou utente e ainda de campos de uma consulta. Adicionalmente, permite ainda que o rececionista tenha acesso a um conjunto de estatísticas extras, como por exemplo, o número de médicos e utentes existentes na clínica, o médico com mais consultas realizadas, entre outras.

De uma forma geral, as *views* *registarutente*, *registarmedico* e *registarfamiliar* permitem o registo de utentes, médicos e familiares, respetivamente. Estas validam os formulários de registo, prosseguindo para extração dos dados utilizando o método *cleaned\_data*. Posteriormente, são feitas também verificações de forma a impedir que haja utilizadores com números de identificação

e *username* iguais no banco de dados. Caso exista, é enviada uma mensagem a informar o utilizador.

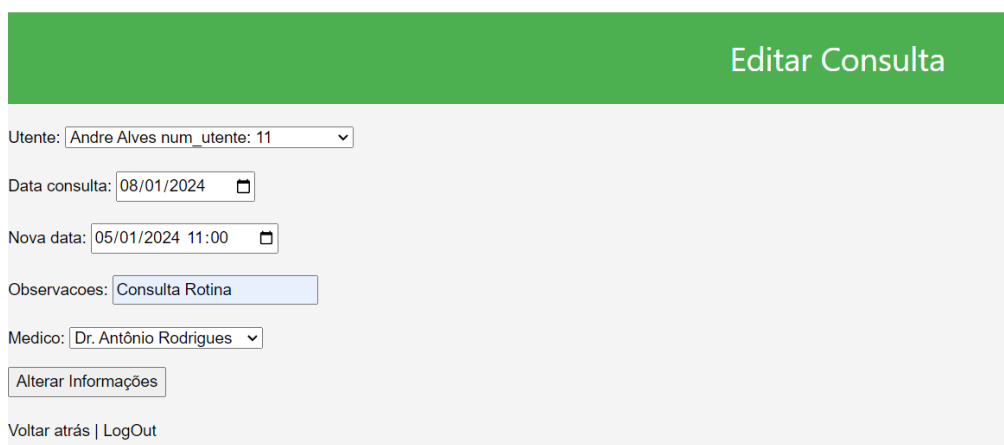
De seguida, após as verificações efetuadas, são criados os objetos pretendidos (Utente, Medico e Familiar) e adicionados ao respetivo grupo.

O processo de registo de uma consulta, cuja página está representada na Figura 15, é semelhante aos anteriores. Inicialmente, o formulário que contém informações como o número do utente, médico, data da consulta e observações é validado. Em seguida, verifica-se se o utente para o qual se pretende registar uma consulta existe e dependendo disso cria-se ou obtém-se a ficha médica associada a ele. De seguida, verifica-se a disponibilidade do médico para a data escolhida e caso haja, cria-se uma nova instância da Consulta.

The screenshot displays a web interface for a receptionist's menu. At the top, a green header bar contains the text 'Menu Rececionista - Registo'. Below this, the main content area has a light gray background. The title 'Adicionar Consultas' is centered at the top of this area. A message 'Consulta registada com sucesso.' is displayed. Below the message, there are several input fields: 'Medico:' with a dropdown menu showing 'Dr. Antônio Rodrigues', 'Data consulta:' with a date and time picker showing '15/01/2024 18:00', 'Observacoes:' with a text box containing 'Consulta Rotina', and 'Num utente:' with a text box containing 'U1'. A button labeled 'Adicionar Consulta' is positioned below these fields. At the bottom of the form, there are two links: 'Voltar ao Menu Rececionista' and 'LogOut'.

**Figura 15-**Adicionar Consultas no Menu Rececionista.

Para além de funcionalidades de registo, o rececionista tem ainda permissões para consultar informações pessoais, familiar associado, exames e consultas por número de utente. Tem, também, acesso aos históricos de consultas e exames realizados na clínica, bem como às consultas agendadas ou dadas para um médico em específico. Adicionalmente, o utilizador rececionista pode realizar alterações em determinados campos numa consulta, tal como mudar a data da consulta para um horário mais adequado, selecionar um médico diferente para a realizar ou alterar as observações.



**Figura 16-**Editar Consulta no Menu Rececionista.

A funcionalidade *Alterar Consulta* no sistema hospitalar permite aos rececionistas realizarem modificações em informações de consultas já agendadas. Recorrendo ao formulário *AlterarCons*, os rececionistas devem fornecer dados como o nome do utente e a data da consulta original. Isto irá permitir a modificação da consulta, possibilitando a introdução de uma nova data, observações adicionais e, se aplicável, a seleção de um médico diferente. Após a validação do formulário, a *view* pesquisa o utente correspondente e a consulta associada, realiza as alterações desejadas e exibe mensagens indicando o sucesso ou falha da operação. Esta funcionalidade é útil para corrigir erros ou ajustar detalhes de consultas no sistema hospitalar. As *views* correspondentes à alteração de informações pessoais de um médico ou utente funcionam de forma semelhante.

Por fim, de forma a proporcionar uma visão abrangente da atividade clínica, foram desenvolvidas algumas estatísticas adicionais. Estas prendem-se com o número total de consultas, utentes, médicos e exames registados. Incluem, ainda, informações sobre o utente com maior número de consultas e identificam também o médico com o maior volume (de consultas), bem como o *top 10* de medicamentos prescritos. Estes dados podem auxiliar na gestão interna e na identificação de padrões relevantes e apresentam-se na Figura 17.



**Figura 17-**Estatísticas adicionais sobre o sistema clínico inerente.

## 5. Conclusões e Trabalho Futuro

Em conclusão, o desenvolvimento da aplicação *DJANGO* para a gestão de uma unidade de saúde apresenta vantagens a nível de eficiência operacional e organizacional. A implementação dos menus para quatro tipos de utilizadores diferentes, cada um com permissões e funcionalidades específicas, proporciona uma abordagem abrangente e segura para a gestão de informações no ambiente da saúde.

Como trabalho futuro, algumas melhorias específicas podem ser consideradas para aprimorar ainda mais a aplicação. Inicialmente, seria possível melhorar otimização das *views* e *templates* de *HTML*, uma vez que foram desenvolvidas funções muito semelhantes para diferentes utilizadores.

Além disso, o aumento do número de funcionalidades poderia ser explorado de forma a abranger uma gama mais ampla de processos relacionados à gestão de saúde, como por exemplo, gestão de *stock* de medicamentos ou análises estatísticas mais detalhadas. Por outro lado, a introdução de novos tipos de utilizadores, como por exemplo, enfermeiros, técnicos e auxiliares de saúde pode enriquecer o sistema, expandindo as capacidades de gestão e atendendo às diferentes funções existentes na unidade de saúde.

Outro ponto crucial para o desenvolvimento futuro da aplicação é a melhoria do agendamento de consultas, através da implementação de um sistema mais sofisticado, considerando, por exemplo, a disponibilidade dos profissionais de saúde.

Em resumo, o trabalho desenvolvido até agora, com auxílio da ferramenta *DJANGO*, estabelece a base para a gestão de uma unidade de saúde, ainda que existam melhorias a levar em conta que permitirão futuramente evoluir e atender à dinamicidade do setor da saúde.